

What's new in JupyterLab 4.6

and the tooling behind it

prototype

lint

build

test

Mike Krassowski



github.com/krassowski/mini-summit

The screenshot displays the JupyterLab 4.6 interface. On the left, a file explorer shows a directory structure with files like 'Lorenz.ipynb', 'lorenz.py', 'Data cleaning.ipynb', 'Analysis.ipynb', and 'README.md'. The main area is split into two panes. The left pane shows the 'Lorenz.ipynb' file, which contains text about the Lorenz Differential Equations, the equations themselves, and code snippets for solving them and plotting histograms. The right pane shows a 'study.csv' file, which is a table with columns 'subject', 'group', 'score', and 'minutes'.

	subject	group	score	minutes
1	S01	A	71.2	34
2	S02	B	64.8	41
3	S03	A	88.1	29
4	S04	B	59.3	45
5	S05	A	77.6	31
6	S06	B	69.0	38
7	S07	A	92.4	27
8	S08	B	61.7	44
9	S09	A	80.3	33
10	S10	B	73.5	36
11	S11	A	85.9	30
12	S12	B	66.2	40
13	S13	A	79.8	32
14	S14	B	70.1	39
15	S15	A	83.0	28
16	S16	B	62.5	43
17	S17	A	90.7	26
18	S18	B	68.4	37
19	S19	A	75.2	35
20	S20	B	63.9	42



Modified



11m ago



19m ago



42m ago

36s ago



2y ago

18m ago



39m ago



2d ago

Left, right, top or bottom: **Settings** › **Interface** › **Activity Bar Position**.

1 of 8

Back

Next

Partial Equations

Before we start, we import some preliminary libraries. We will also import (below) the accompanying `lorenz.py` file, which contains the actual calculation and plotting routine.

[1]:

```
1 %matplotlib inline
2 from ipywidgets import interactive, fixed
```

Equations

We explore the Lorenz system of differential equations:

$$\dot{x} = \sigma(y - x)$$

$$\dot{y} = \rho x - y - xz$$

$$\dot{z} = -\beta z + xy$$

Interactive plot

Name	Modified
data	11m ago
experiments	19m ago
tests	42m ago
Lorenz.ipynb	36s ago
lorenz.py	2y ago
Data cleaning.ipynb	18m ago
Analysis.ipynb	39m ago
README.md	2d ago

The Lorenz Differential Equations

Before we start, we import some preliminary libraries. We will also import (below) the accompanying `lorenz.py` file, which contains the actual solver and plotting routine.

```
[1]: 1 %matplotlib inline
      2 from ipywidgets import interactive, fixed
```

Equations

We explore the Lorenz system of differential equations:

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= \rho x - y - xz \\ \dot{z} &= -\beta z + xy\end{aligned}$$

Interactive plot

Let's change (σ, β, ρ) with ipywidgets and examine the trajectories.

```
[2]: 1 from lorenz import solve_lorenz
      2 w = interactive(solve_lorenz, sigma=(0.0, 50.0), rho=(0.0, 50.0))
      3 w
```

Histograms

```
[3]: 1 t, x_t = w.result
      2 xyz_avg = x_t.mean(axis=1)
      3 plot_histograms(xyz_avg)
```

The averages of x , y and z settle near the two attractors.

	subject	group	score	minutes
1	S01	A	71.2	34
2	S02	B	64.8	41
3	S03	A	88.1	29
4	S04	B	59.3	45
5	S05	A	77.6	31
6	S06	B	69.0	38
7	S07	A	92.4	27
8	S08	B	61.7	44
9	S09	A	80.3	33
10	S10	B	73.5	36
11	S11	A	85.9	30
12	S12	B	66.2	40
13	S13	A	79.8	32
14	S14	B	70.1	39
15	S15	A	83.0	28
16	S16	B	62.5	43

Go back to the cell you last edited

Two new buttons in the Table of Contents move back and forward through the cells you edited.

File

Edit

View

Run

Kernel

Tabs

Settings

Help

+

/ metanalysis / experiments /

Name

baseline.ipynb

2h ago

sweep-rho.ipynb

19m ago

sweep-sigma.ipynb

1h ago

params.json

1h ago

results.csv

19m ago

NOTES.md

3d ago

Modified

Lorenz.ipynb

+

Code

Python 3 (i

ential Equations

the preliminary libraries. We will also i

(below) the accompanying `lorenz.py` file, which contains the actual

and plotting routine.

[1]:

1 %matplotlib inline

2 **from** ipywidgets **import** interactive, fixed

Equations

We explore the Lorenz system of differential equations:

$\dot{x} = \sigma(y - x)$

$\dot{y} = \rho x - y - xz$

$\dot{z} = -\beta z + xy$

Interactive plot

Type a path, press Tab

Click the breadcrumbs to edit the path. Tab completes folder names.

5 of 8

Back

Next

Lorenz.ipynb



Paused in Debugger



Debugger buttons next to your code

A small panel with Continue, Step and Stop floats over the notebook. Sources now open in the main area.

7 of 8

[Back](#)

[Next](#)

The Lorenz Differential Equations

Before we start, we import some preliminary libraries. We will use (below) the accompanying `lorenz.py` file, which contains the solving and plotting routine.

```
[1]: 1 %matplotlib inline
      2 from ipywidgets import interactive, fixed
```

Equations

We explore the Lorenz system of differential equations:

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= \rho x - y - xz \\ \dot{z} &= -\beta z + xy\end{aligned}$$

Interactive plot

Let's change (σ, β, ρ) with ipywidgets and examine the trajectories.

```
[2]: 1 from lorenz import solve_lorenz
      2 w = interactive(solve_lorenz, sigma=(0.0, 50.0), rho=(0.0, 50.0))
      3 w
```

Histograms

			core	minutes
			71.2	34
			64.8	41
			88.1	29
			59.3	45
			77.6	31
6	S06	B	69.0	38
7	S07	A	92.4	27
8	S08	B	61.7	44
9	S09	A	80.3	33
10	S10	B	73.5	36
11	S11	A	85.9	30
12	S12	B	66.2	40
13	S13	A	79.8	32
14	S14	B	70.1	39
15	S15	A	83.0	28
16	S16	B	62.5	43
17	S17	A	90.7	26
18	S18	B	68.4	37
19	S19	A	75.2	35
Table of Contents			63.9	42

LORENZ.IPYNB



The Lorenz Differential Equations

Equations

Interactive plot

♥ BEFORE WE START

This talk is also a report, and a thank-you



FUNDED

four community proposals,
in the Foundation's first
funding round



DELIVERED

the four proposals, working
together



THE COMMUNITY

reviewed, triaged, tested and
adopted the work, and
reported problems



TRAVEL

OpenTeams paid for my trip
to this summit

📄 Progress reports: jupyter-governance.github.io/funding-proposals

🕒 JUPYTERLAB 4.6 • RELEASED 18 JUNE 2026

One of the largest minor releases

68

new features and enhancements

97

bug fixes

95

contributors, 39 of them first-time

38

documentation improvements

171

maintenance tasks, a record

Also in

📖 Notebook 7.6

🌐 JupyterLite 0.8

Among minor releases, only 3.1 merged more pull requests.

A scratchpad console

Try code on the same kernel without adding cells to the notebook.

Ctrl + B

The image shows a JupyterLab interface with a notebook on the left and a scratchpad console on the right. The notebook, titled 'analysis', contains two code cells. The first cell defines 'months' and 'temperatures' lists. The second cell calculates the average temperature and finds the warmest month, outputting 'Average temperature: 22.0°C' and 'Warmest month: Jul'. The scratchpad console, titled 'Python 3 (ipykernel)', shows the same code being executed, with the output of the second cell being a list of tuples: [(26.1, 'Jul'), (25.7, 'Aug'), (24.9, 'Jun')].

Sensor data analysis

Explore the monthly temperature readings collected by the weather station.

```
[1]: months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
    temperatures = [18.2, 19.1, 20.3, 21.8, 23.4, 24.9, 26.1, 25.7, 24.2, 22.0, 19.8, 18.5]
```

```
[2]: average = sum(temperatures) / len(temperatures)
    warmest = months[temperatures.index(max(temperatures))]
    print(f"Average temperature: {average:.1f}°C")
    print(f"Warmest month: {warmest}")
```

Average temperature: 22.0°C
Warmest month: Jul

```
Python 3.14.0 (main, Nov 19 2025, 23:04:28) [Clang 21.1.4 ]
Type 'copyright', 'credits' or 'license' for more
information
IPython 9.11.0 -- An enhanced Interactive Python. Type '?'
for help.
Tip: The `%timeit` magic has a `-o` flag, which returns the
results, making it easy to plot. See `%timeit?`.

[3]: sorted(zip(temperatures, months), reverse=True)[:3]
[3]: [(26.1, 'Jul'), (25.7, 'Aug'), (24.9, 'Jun')]
```

Also in 7.6: “Close and Shut Down” asks before it stops the kernel.

🌟 ALSO IN 4.6

And more



Shortcuts for any command

Add them in the shortcut editor. No JSON needed.



Find and Replace in the Edit menu

`Ctrl` `H`, and `Ctrl` `Y` to redo on Windows and Linux.



Language packs from Settings

“Install more languages...” opens the Extension Manager.



Ghost text with highlighting

Inline completions are coloured like code, at every cursor.



Zoom one panel

`Ctrl` + mouse wheel. Other panels do not change.



Split a tab four ways

Left, right, up or down.



Keyboard and screen readers

The terminal no longer keeps the keyboard focus. Focus is easier to see.



Open in Terminal

From any folder in the file browser.



Fewer mistakes

JupyterLab asks before it deletes a cell. Keys 1 to 6 no longer turn code into markdown.

Faster builds, fewer dependencies

~5x

faster builds

JupyterLab, Notebook and extensions moved from webpack to Rspack.



Build an extension without installing JupyterLab

The build tooling now lives in `jupyter-builder`.



Jupyter-specific lint rules in the extension template

`@jupyter/eslint-plugin` is on by default for new extensions.

PART 2

How did we get here?

Four proposals funded by the Jupyter Foundation.



Prototype

Plugin Playground



Lint

@jupyter/eslint-plugin



Build

jupyter-builder



Test

UI tests

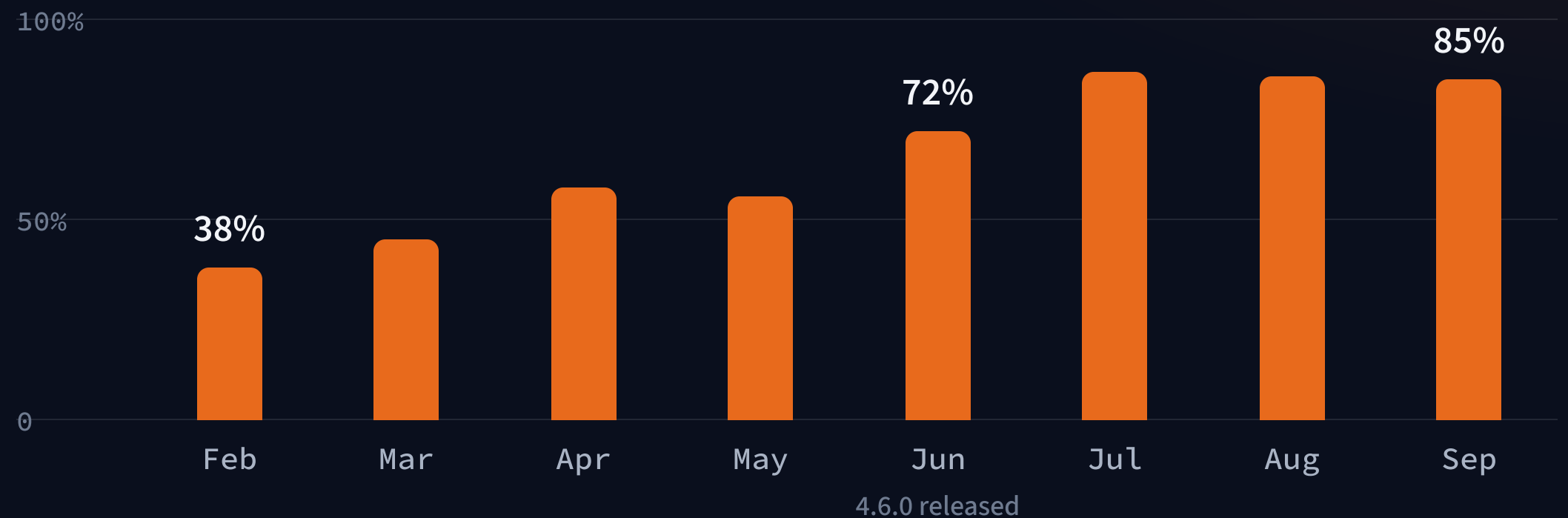
Writing code became cheap

52%

of JupyterLab 4.6 pull requests that answered the AI question say **AI generated some of the content**

87 of 168 · the PR template asks since February 2026

Share of PRs with AI-generated content, by month opened



28 of 30 merged PRs that changed a UI test from June to September were AI-assisted

39 first-time contributors to JupyterLab 4.6

Still expensive:

🔧 prototyping

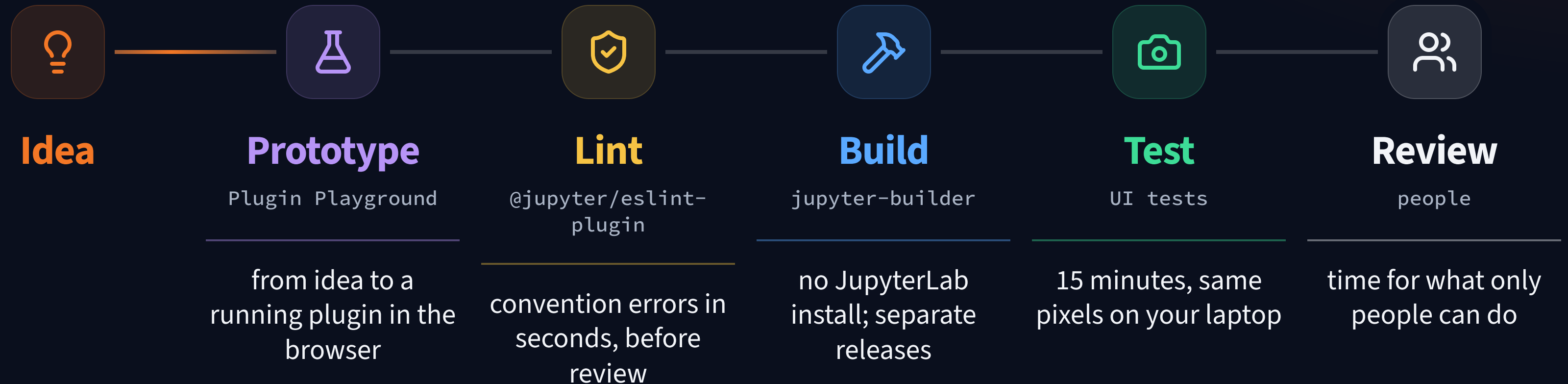
👥 reviewing

🔗 building

📷 testing

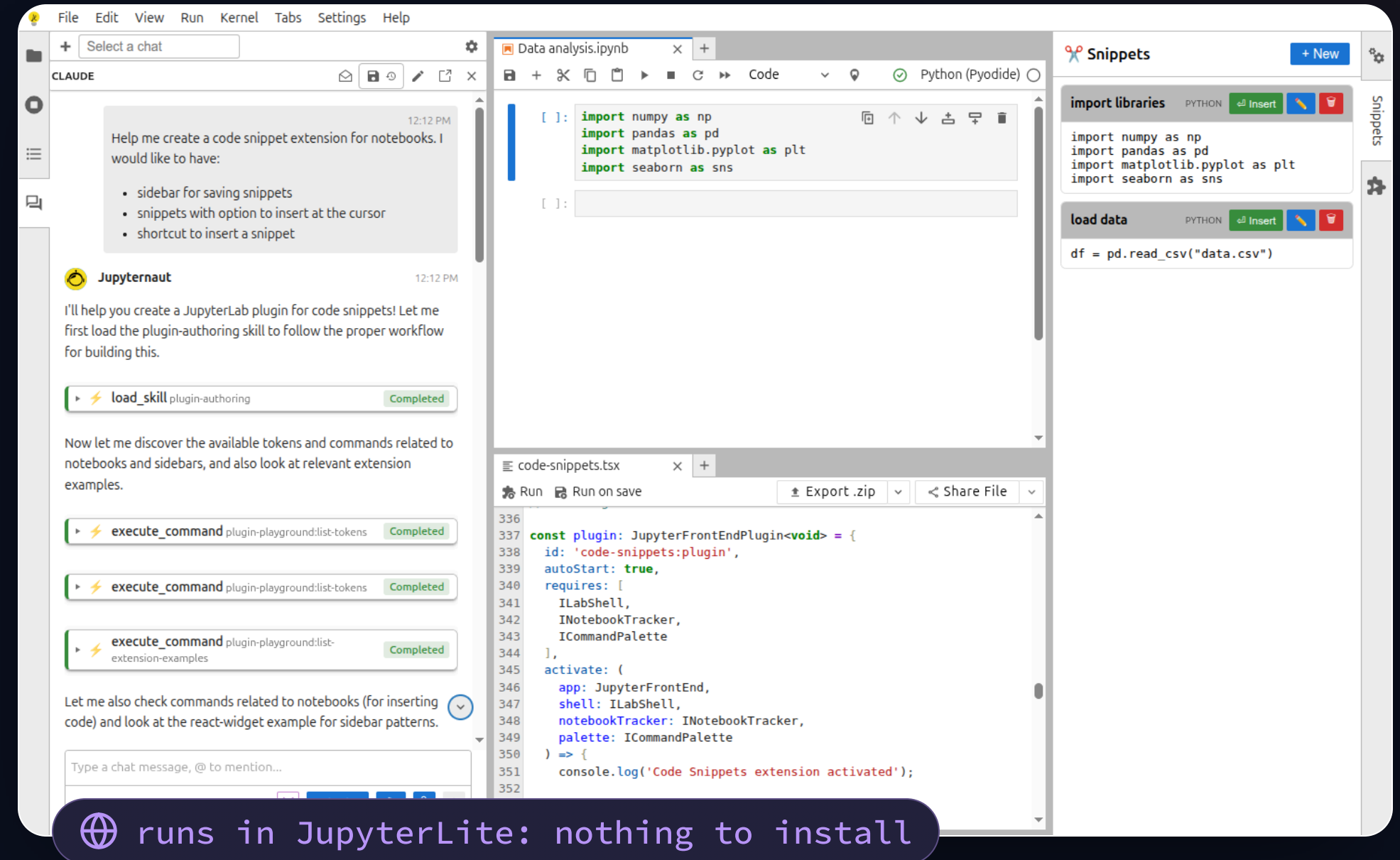
FROM IDEA TO RELEASE

One pipeline, four proposals



Plugin Playground 1.0

- 🔧 **Describe it** to the AI, or start from a file
- 🔗 **Insert** tokens and commands from the sidebar
- ▶️ **Load** the file as an extension, no build
- 🔄 **Run on save** while you iterate
- 🔗 **Share** a link for review
- 📦 **Export** a wheel or a zip for a real repository



🌐 runs in JupyterLite: nothing to install

Built by  **Anuj Singh** (OpenTeams). Design clinic by  **Smera Goel** (Quansight).

`pip install jupyterlab-plugin-playground`

How AI helps in the playground



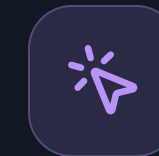
No server needed

Built on `jupyterlite/ai`, so it works in JupyterLite and on Binder too.



A guide for writing plugins

The AI follows written instructions, so first drafts have the structure that the playground expects.



The same commands as you

The AI runs JupyterLab commands: create, load, export, share.

🔗 We sent four improvements to `jupyterlite/ai`. Next: code completion and error checking (LSP) in JupyterLite.

@jupyter/eslint-plugin

8 → 23

rules, from 1.0 in May to 1.3 in September

Used by

extension template

jupyter-collaboration

jupyterlab-git

jupyter-chat

ipyparallel



Plugins

plugin-activation-args

4



Translations

no-untranslated-string

4



Memory leaks

require-signal-cleanup

5



UI tests

galata-prefer-notebook-cell-helper

6



Startup and deployment

prefer-lazy-imports

2



Commands and settings

no-schema-enum

2

Led by  **Darshan Paudyal** (OpenTeams).

```
npm i -D @jupyter/eslint-plugin
```

Agents can skip AGENTS.md. They cannot skip a lint error.

AGENTS.md

- **68%** of instructions followed when there are 500 of them
- **+20%** cost, and no better results in general
- When an agent skips a line, nothing fails

Sometimes followed, and no error when it is not

A lint rule

ui-tests/tests/run-cell.spec.ts

12:3 **error** Use `page.notebook.setCell()` / `runCell()`
instead of raw cell selectors
`jupyter/galata-prefer-notebook-cell-helper`

Runs every time. Names the file, the line and the fix.

The agent does not need to know the rule. The linter tells it.

When should you write a custom lint rule?

- 1 TypeScript cannot check the convention
- 2 It is rare in public code, so models have not learned it
- 3 Breaking it causes no error
- 4 The error message can say what to do instead

Wrote the same review comment three times? **That comment is your rule.**

AGENTS.md

```
## Before you finish
- Run `jlp eslint` and fix every
  `jupyter/*` error.
```

jupyter-builder

BEFORE

 your extension



needs to build

 all of JupyterLab

server, application, every core package

↔ Packagers: `jupyterlab_pygments` needs JupyterLab to build,
and JupyterLab includes it.

NOW

 your extension



needs to build

 jupyter-builder

PyPI and npm



@jupyterlab/core-meta

compatibility metadata

```
- jupyter labextension build .  
+ jupyter-builder build . # --core-version 4.4.x
```

Led by  **Darshan Paudyal** (OpenTeams). Started in GSoC 2024 by Ronan Coutinho, mentored by Frédéric Collonval.

A fix in 24 hours, without a JupyterLab release



Rspack 2.0

The new bundler switched to Module Federation v2 by accident.



Report

Extensions that use older shared packages break. The jupyter-chat maintainers report it.



Under 24 hours

A jupyter-builder minor release goes back to Module Federation v1.



For everyone else

No JupyterLab release needed. Extensions upgrade when they are ready.

The build system now has its own releases.

UI tests you can trust

	BEFORE	NOW
Full UI test run	55 min	15 min
New reference images	45 min, maintainers only	1 min, anyone
Flaky tests per run	17	2.4
Runs with a hard failure	54%	14%
Same pixels on your laptop	✗ no	✓ yes, with the CI fonts

 Fixing flaky tests found **19 real bugs** in JupyterLab. All are fixed.

Agents need fast and reliable tests

55 min per run



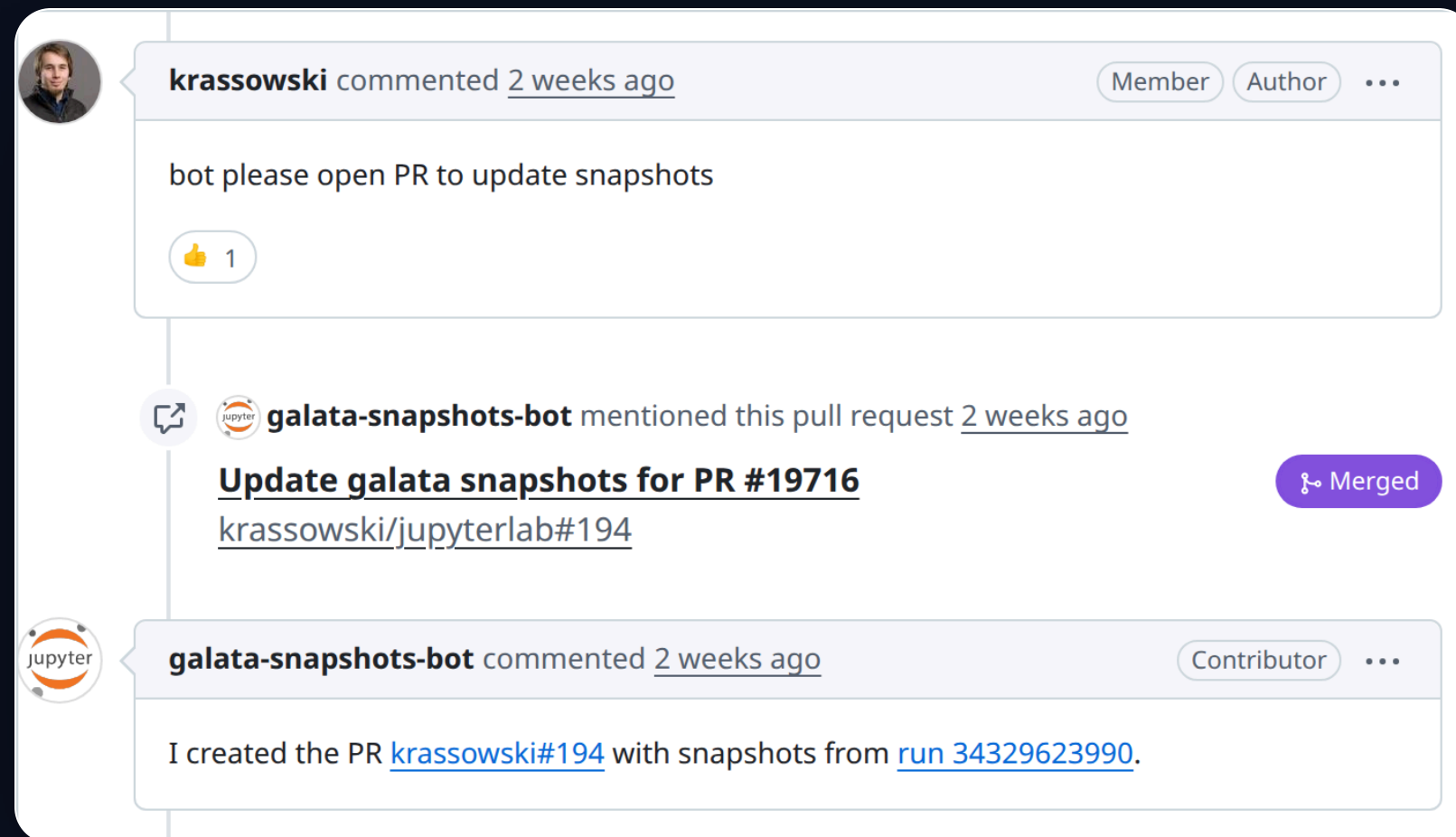
15 min per run



rounds of change and check in a working day

- ✓ A failure on your laptop is a real failure. No retries hide flaky tests, so agents stop adding fixed waits.
- ✓ The rules are in files that agents read: AGENTS.md, the contributing guide, lint rules.
- ✓ Every new or changed test has a video in the report.

Anyone can update snapshots



krassowski commented [2 weeks ago](#) Member Author ...

bot please open PR to update snapshots

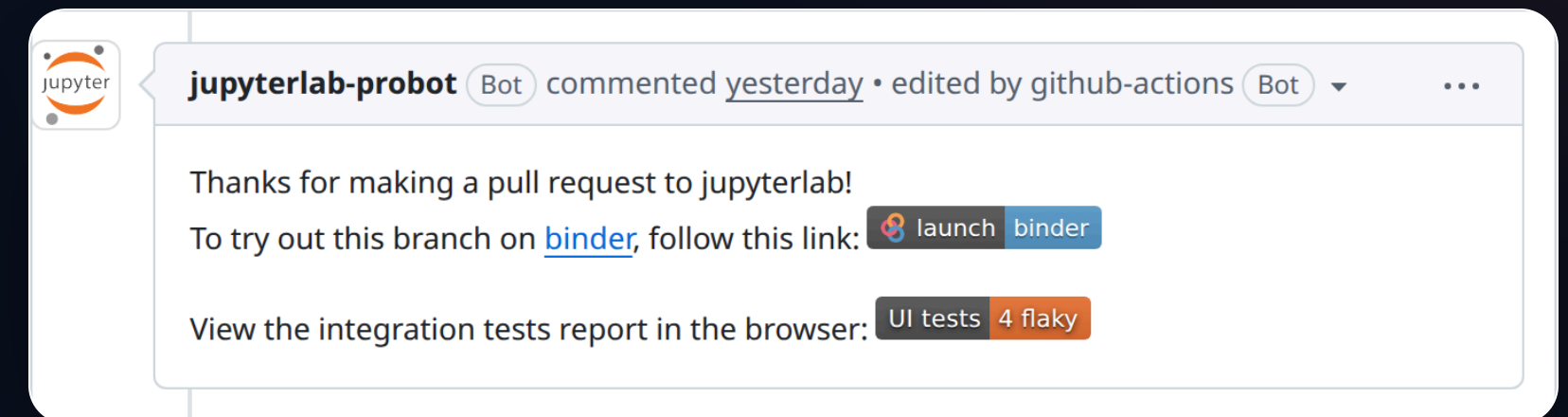
👍 1

galata-snapshots-bot mentioned this pull request [2 weeks ago](#)

Update galata snapshots for PR #19716 🔗 Merged
[krassowski/jupyterlab#194](#)

galata-snapshots-bot commented [2 weeks ago](#) Contributor ...

I created the PR [krassowski#194](#) with snapshots from [run 34329623990](#).



jupyterlab-probot Bot commented [yesterday](#) • edited by github-actions Bot ...

Thanks for making a pull request to jupyterlab!

To try out this branch on [binder](#), follow this link: [launch binder](#)

View the integration tests report in the browser: [UI tests 4 flaky](#)

The badge opens the Playwright report in the browser. No download, no unzip.

Comment **please open PR to update snapshots**: a bot opens a PR with the new images in about a minute.



Safe for pull requests from forks: the bot commits to its own fork, a maintainer approves each run, and zizmor checks the workflow files.

Use it in your project

Plugin Playground

```
pip install jupyterlab-plugin-playground
```

Or open it in JupyterLite, no install.

@jupyter/eslint-plugin

```
npm i -D @jupyter/eslint-plugin
```

Then add `configs.recommended`. Run it once on your extension.

jupyter-builder

```
pip install jupyter-builder
```

`jupyter-builder build .` instead of `jupyter labextension build .`

inline-playwright-report

```
jupyterlab/maintainer-tools
```

Turns a Playwright report into one HTML file that GitHub can show.

ui-test-report-comment

```
jupyterlab/maintainer-tools
```

Posts the badge comment, with safety checks for forks.

Stable screenshots

```
--disable-lcd-text
```

Pin fonts with `@fontsource`. Set `fullyParallel: true` so shards get equal work.

New challenges with AI, and our answers

CHALLENGE

WHAT WE DO



Code arrives faster than people can review it



Move feedback from people to tools: lint, badges, snapshot PRs on request



Agents skip written conventions



Lint rules whose error message says how to fix it



Flaky tests mislead agents



Tests that give the same result on a laptop; a weekly report of flaky tests



More first-time contributors



Workflows that need no maintainer; an AI usage question in the PR template



Bots that act on forks can be abused



No write access in jobs that read fork content; maintainer approval; zizmor checks

Four proposals, delivered



Improve visual regression testing #7

55 to 15 min; snapshots on request; local runs match CI

✓ COMPLETE



Plugin Playground #8

1.0 with AI; JupyterLite and Binder; share and export

✓ COMPLETE



Finish the build system separation #9

jupyter-builder 1.0 to 1.2; Rspack; no JupyterLab needed

✓ COMPLETE



Custom linting #11

23 rules; in the extension template; stricter JupyterLab CI

✓ COMPLETE

Delivered by



Reports: jupyter-governance.github.io/funding-proposals

Thank you

Questions?



Slides, sources and every link

github.com/krassowski/mini-summit



BUILT BY • OPENTEAMS



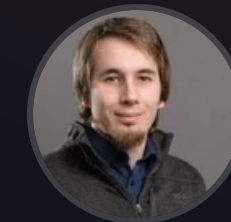
Anuj Singh

@MUFFANUJ
Plugin Playground



Darshan Paudyal

@Darshan808
lint, build



Mike Krassowski

@krassowski
UI tests, guidance

ADVICE AND SUPPORT • QUANSIGHT



Smera Goel

@smeragoel



Matthias Bussonnier

@Carreau



Tania Allard

@trallard

ADMINISTRATIVE SUPPORT

Erika Oliphant (OpenTeams), Ashley Baal (Quansight)

REVIEWS AND FEEDBACK

Jeremy Tuloup, Nicolas Brichet, Nicholas Bollweg, Jason Grout, Yann Pellegrini, Florence Haudin, Matt Fisher, Frédéric Collonval

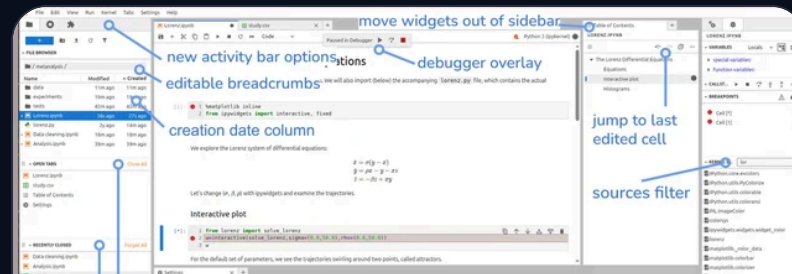
AND

95 contributors to JupyterLab 4.6, the weekly triage group, and everyone who tried the pre-releases

FURTHER READING

The posts behind this talk

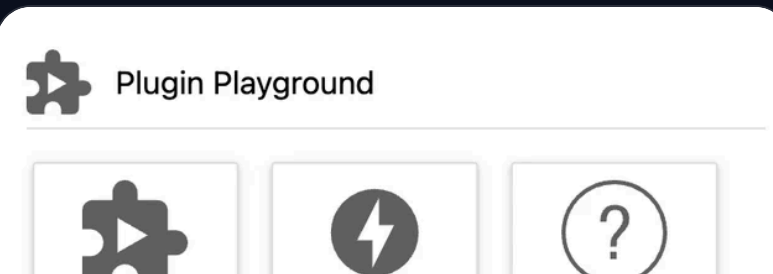
JUPYTER BLOG



● RELEASE

JupyterLab 4.6 and Notebook 7.6 are out!

Michał Krassowski · 2 July 2026



● PROTOTYPE

A User's Journey with Plugin Playground: From First Idea to Installable JupyterLab Extension

Anuj Singh · 28 May 2026



● LINT

Catching Jupyter-specific bugs before CI does: announcing jupyter eslint plugin

Darshan Paudyal · 11 June 2026

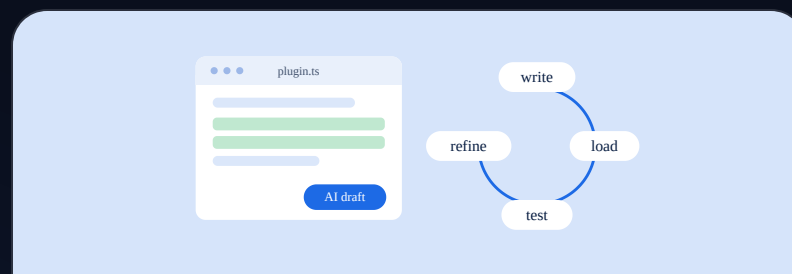


● BUILD

Announcing jupyter-builder: A Standalone Build System for JupyterLab Extensions

Darshan Paudyal · 19 June 2026

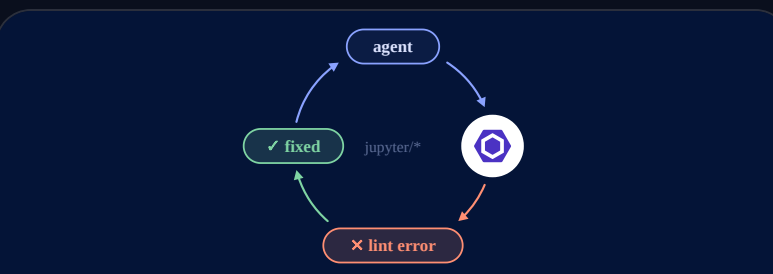
OPENTEAMS ENGINEERING BLOG



● PROTOTYPE

Plugin Playground AI Integration for Faster Plugin Prototyping

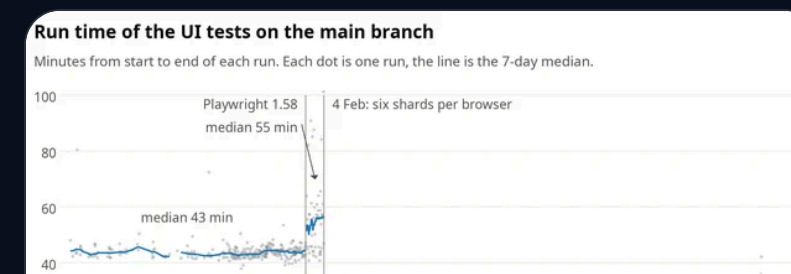
Anuj Singh · 24 April 2026



● LINT

Your AI Agent Ignored AGENTS.md. Your Linter Won't Let It.

Darshan Paudyal · 1 October 2026



● TEST

Reliable Visual Regression Testing for Humans and Coding Agents

Mike Krassowski · October 2026



Every link, the slides and their sources

[github.com/
krassowski/
mini-summit](https://github.com/krassowski/mini-summit)

AI usage in JupyterLab pull requests

MONTH OPENED (2026)	SAID YES	ANSWERED	SHARE	RELEASE	SAID YES	ANSWERED	SHARE
February (from the 13th)	10	26	38%	4.6.0	87	168	52%
March	19	42	45%	After 4.6.0	87	106	82%
April	21	36	58%	The question “Some or all of the content of this PR was generated by AI” is in the PR template since 13 February 2026. PRs by bots, and PRs that did not answer, are not counted. Script: <code>tools/ai_usage.py</code> .			
May	29	52	56%				
June	33	46	72%				
July	33	38	87%				
August	18	21	86%				
September	11	13	85%				

The 23 rules of @jupyter/eslint-plugin 1.3

Plugins

plugin-activation-args
plugin-description
plugin-id-convention
token-format

Memory leaks

require-signal-cleanup
require-signal-this-arg
prefer-signal-this-arg
require-disposable-ownership
require-disposable-transfer

Commands and settings

command-described-by
no-schema-enum

UI tests

galata-prefer-notebook-cell-helper
galata-prefer-filebrowser-helper
galata-prefer-menu-helper
galata-prefer-context-menu-helper
galata-prefer-sidebar-activity-helper
require-soft-assertions-before-snapshots

Translations

no-untranslated-string
no-translation-concatenation
no-dynamic-translation
incorrect-translator-usage

Startup and deployment

prefer-lazy-imports
no-pageconfig-base-url

Screenshots on other systems

	BEFORE	NOW
Fedora 44, default fonts: screenshots that match CI	1%	85%
Fedora 44, default fonts: tests that pass	56%	93%
Ubuntu 26.04 runner: screenshots that match CI	35%	100%
Ubuntu 26.04 runner: tests that pass	71%	99.8%

Measured on GitHub Actions on 26 September 2026. With the CI font packages installed, Fedora 44 and openSUSE Tumbleweed match all 255 comparisons. The remaining Fedora differences are Mermaid diagrams, which draw text with system fonts.